

Testing Scientific Expectations under Relational Event Models with BFpack

Joris Mulder

2026-07-29

How to use this document. Every code block below is ready to paste into an R session and run in order. To keep the handout light, the chunks are *not* executed while knitting (`eval = FALSE` in the setup chunk); instead the expected results are shown and discussed in the text. To run the analysis live, either flip `eval = FALSE` to `eval = TRUE`, or simply copy each block into R. The single full-sequence fit is fast; the moving-window loop refits the REM many times and takes a few minutes.

*This is the relational-event companion to the ERGM tutorial. It reuses the same four-step **BFpack** workflow — fit, get names, formulate constraints, test — on a relational event model (REM) of militarized interstate disputes. The only real difference is the last step: for a REM we call `BF()` on the **estimates and their covariance matrix** (`BF.default`) rather than on a fitted model object.*

Introduction and learning objectives

A **relational event model (REM)** describes a stream of time-stamped, directed events — *who* acts on *whom*, and *when* — through the *rate* at which each possible dyad produces the next event. This tutorial teaches **Bayesian hypothesis testing with Bayes factors and posterior probabilities** for REM coefficients, using the **BFpack** package together with the **remverse** (`remify`, `remstats`, `remstimate`).

Why bother going beyond the *p*-value? The same three arguments from the ERGM tutorial apply — a *p*-value cannot support a null, it is inconsistent under the null, and it cannot directly test order constraints — but REMs add a bonus:

Because the sample size of a REM is the **number of events** (not the number of nodes), evidence genuinely accumulates as we observe the sequence for longer. A Bayes factor is *consistent*, so “collect more events and update the Bayes factor” is a legitimate and practical strategy — and, as we will see, we can even track how the evidence **changes over time**.

By the end you will be able to:

- fit a tie-oriented REM with `remstats + remstimate` and read off the exact coefficient names with `coef()`;
 - translate a substantive expectation into a **BFpack** hypothesis string using equality (=) and order (>, <) constraints;
 - compute Bayes factors and posterior probabilities with `BF()` via the `BF.default` route (`estimates + covariance matrix + n`);
 - run a **moving-window** analysis to see how the evidence for competing hypotheses evolves across the observation period; and
 - adapt the workflow to your own constrained hypotheses (the exercise).
-

Setup: packages and data

```
# Install once, if needed:
# install.packages("remverse")

library("remverse")
library("BFpack")      # Bayes factors for constrained hypotheses
```

The empirical case is the sequence of **militarized interstate disputes (MIDs)** from the Correlates of War project (Singer, 1972): a directed event $i \rightarrow j$ means state i initiated a militarized action against state j . The workshop data folder ships one edgelist plus a set of (time-varying) country covariates as CSV files.

```
# Point this at the folder where you unzipped the workshop data.
data_dir <- "dir"
cov <- function(f) read.csv(file.path(data_dir, f), stringsAsFactors = FALSE)

# Edgelist: one row per event, ordered in time.
el <- cov("edgelist_full.csv")
el$time <- as.Date(el$time)
el <- el[order(el$time), ]
```

The sequence contains roughly **4,670 directed events** at **2,429 unique time points** (several disputes can fall on the same date), spanning the post-war period. A relational event analysis of these data was presented by Lakdawala et al. (2026).

Build the relational event history

`remify()` turns the raw edgelist into a relational event history object. We use a **tie-oriented** model with an *active-saturated* risk set (every ordered pair of observed actors is at risk at every time point).

```
reh_tie <- remify(el[, c("time", "actor1", "actor2")],
                 directed = TRUE, model = "tie",
                 time_units = "years", riskset = "active_saturated")
reh_tie      # a short summary; reh_tie$M is the number of unique time points
# plot(reh_tie) # optional: event-rate diagnostics
```

Actor covariates, including the time-varying regime dummies

The covariates are read as data frames on `remify`'s time scale. The two we will test are **regime-type dummies**, built from each state's Polity score *dem* in each year, so they **vary over time**:

```
origin <- min(el$time)      # first event = time 0
to_years <- function(d) as.numeric(as.Date(d) - origin) / 365.25

ac <- list(major = cov("cov_major.csv"),
           dem = cov("cov_dem.csv"),
           religion = cov("cov_religion.csv"))

maj <- ac$major              # 'major' (0/1)
maj$time <- to_years(maj$date)

reg <- ac$dem
reg$time <- to_years(reg$date)
reg$democracy <- as.integer(reg$dem >= 6)      # democracy dummy
reg$autocracy <- as.integer(reg$dem <= -6)     # autocracy dummy
```

```
rel <- ac$religion # 'islam' / 'christianity'
rel$time <- to_years(rel$date)
```

Read the dummies carefully. democracy = 1 when Polity ≥ 6 , autocracy = 1 when Polity ≤ -6 ; everything in between — the *anocracies* / mixed regimes — is the **baseline**. So `send_democracy` and `send_autocracy` are each measured *relative to that baseline*, and because Polity changes over the decades a state can move between categories across the sequence.

A tie-oriented REM: the four-step workflow

Step 1 — Fit the full model

The `send()` effects capture how an attribute of the **initiator** shifts its propensity to start a dispute; `inertia` and `reciprocity` are endogenous controls (standardized).

```
tie_effects <- ~
  inertia(scoring = "std") + reciprocity(scoring = "std") + # controls
  send("major",      attr_actors = maj) + # -> send_major
  send("autocracy",   attr_actors = reg) + # -> send_autocracy
  send("democracy",   attr_actors = reg) + # -> send_democracy
  send("islam",       attr_actors = rel) + # -> send_islam
  send("christianity", attr_actors = rel)  # -> send_christianity

training <- 500 # burn-in for the endogenous stats
stats_tie <- remstats(reh_tie, tie_effects = tie_effects, first = training)
fit_tie <- remestimate(reh_tie, stats_tie)
summary(fit_tie)
```

Step 2 — Read the exact parameter names

This step is essential. The names BFPack expects in a hypothesis string are exactly the names printed by `coef()`.

```
est <- coef(fit_tie)
print(est)
#> Effect names include, among others:
#> baseline, inertia, reciprocity,
#> send_major, send_autocracy, send_democracy, send_islam, send_christianity
```

Step 3 — Formulate competing constrained hypotheses

The two effects of interest are `send_democracy` and `send_autocracy`. Each competing hypothesis encodes a substantive theory from international relations about how regime type drives who *initiates* conflict:

- **H1 (null):** `send_democracy` = `send_autocracy` = 0 — *regime type is irrelevant*; initiation is driven by other factors.
- **H2 (democratic peace vs. autocratic aggression):** `send_democracy` < 0 < `send_autocracy` — democracies initiate less, autocracies more.
- **H3 (autocratic aggression only):** `send_democracy` = 0 < `send_autocracy` — democracies behave like the baseline; autocracies stand out.
- **H4 (democratic restraint only):** `send_democracy` < 0 = `send_autocracy` — democracies are distinctively peaceful; autocracies look like the baseline.

`BF()` adds the **complement** (none of the above) automatically, so we compare five hypotheses in total.

```
hyp_gov <- "send_democracy = send_autocracy = 0;
           send_democracy < 0 < send_autocracy;
           send_democracy = 0 < send_autocracy;
           send_democracy < 0 = send_autocracy"
```

Step 4 — Compute Bayes factors and posterior probabilities

For a REM there is no dedicated `BF()` method, so we use the general `BF.default` route: supply the estimates `x`, their covariance matrix `Sigma`, and the number of observations `n`. `BFpack` then builds an *adjusted fractional Bayes factor* using a Gaussian approximation to the likelihood (Gu et al., 2017) — the same fast, approximate Bayes factor used for the ERGM’s speed route.

```
BF1 <- BF(x = est, Sigma = fit_tie$vcov, n = fit_tie$df.null,
          hypothesis = hyp_gov)
BF1
BF1$PHP_confirmatory
summary(BF1) # adds the exploratory Pr(=0 / <0 / >0) table for every parameter
```

Reading the result

The confirmatory posterior probabilities are:

Posterior probabilities of the hypotheses

	Pr(hypothesis data)	
H1	0.000	# regime type irrelevant (null)
H2	0.620	# democracy < 0 < autocracy
H3	0.379	# democracy = 0 < autocracy
H4	0.000	# democracy < 0 = autocracy
H5	0.001	# complement

Almost all posterior mass sits on **H2 (.62)** and **H3 (.38)**. Both agree that **autocracies initiate more** than the baseline; the data are *split* on whether democracies are *additionally* restrained (H2) or simply *indistinguishable from the baseline* (H3). The Bayes factor between them is only about **1.6** in favor of H2 — positive but far from decisive. The null H1 and the “restraint-only” theory H4 are effectively ruled out.

The exploratory part of `summary(BF1)` tells the same story at the parameter level: `send_autocracy` is confidently positive, whereas `send_democracy` splits its mass between “= 0” and “< 0” — which is exactly why H2 and H3 both survive.

Changing the prior probabilities of hypotheses

Equal prior probabilities are the default. To encode differing a-priori plausibility, pass one weight per hypothesis (including the complement) via `prior.hyp.conf`:

```
BF1b <- BF(x = est, Sigma = fit_tie$vcov, n = fit_tie$df.null,
           hypothesis = hyp_gov,
           prior.hyp.conf = c(1, 1, 1, 1, 2)) # more prior mass on the complement
BF1b$PHP_confirmatory
```

The Bayes factors (the evidence *in the data*) do not change; only the posterior probabilities, which combine evidence with the prior odds, shift.

Does the evidence hold over time? A moving-window analysis

The test above averages over the whole post-war period. Because a REM's sample size is the number of *events*, we can instead slide a **window** across the sequence, refit the REM in each window, and watch the posterior probabilities evolve (Mulder & Leenders). This turns a single verdict into a *time series of evidence*.

```
length.window <- 500                                # events per window
M <- reh_tie$M                                       # number of unique time points
steps <- seq(0, M - training - length.window, by = 100)

php_mat <- matrix(NA_real_, nrow = length(steps), ncol = 5)
est_dem_w <- rep(NA_real_, length(steps))
est_auto_w <- rep(NA_real_, length(steps))
mid_time <- as.Date(rep(NA, length(steps)))
utimes <- sort(unique(el$time)) # calendar date of each time point

for (i in seq_along(steps)) {
  step <- steps[i]
  stats_w <- remstats(reh_tie, tie_effects = tie_effects,
                     first = training + step,
                     last = training + step + length.window)
  fit_w <- remestimate(reh_tie, stats_w)

  est_w <- coef(fit_w) # use THIS window's estimates
  est_dem_w[i] <- est_w["send_democracy"]
  est_auto_w[i] <- est_w["send_autocracy"]

  BF_w <- tryCatch(
    BF(x = est_w, Sigma = fit_w$vcov, n = fit_w$df.null, hypothesis = hyp_gov),
    error = function(e) NULL)
  if (!is.null(BF_w)) php_mat[i, ] <- BF_w$PHP_confirmatory

  mid_idx <- round(training + step + length.window / 2)
  mid_time[i] <- utimes[mid_idx] # window mid-point as a date
}

colnames(php_mat) <- c("H1: none = 0", "H2: dem<0<auto",
                      "H3: dem=0<auto", "H4: dem<0=auto", "complement")
```

Plot the posterior probabilities against the window mid-point date, and — so the shifts can be read against the effect sizes that drive them — the underlying estimates on the same time axis:

```
par(mar = c(1, 1, 1, 1))
par(mfrow = c(2, 1))

matplot(mid_time, php_mat, type = "l", lty = 1, lwd = 2, col = seq_len(5),
        ylim = c(0, 1), xaxt = "n",
        xlab = "Window mid-point (year)", ylab = "Posterior probability",
        main = sprintf("PHPs over a moving window (%d events)", length.window))
axis.Date(1, at = pretty(mid_time), format = "%Y")
legend("topleft", legend = colnames(php_mat), col = seq_len(5),
      lty = 1, lwd = 2, cex = 0.8, bty = "n")

matplot(mid_time, cbind(est_dem_w, est_auto_w), type = "l", lty = 1, lwd = 2,
        col = c("blue", "red"), xaxt = "n",
```

```

xlab = "Window mid-point (year)", ylab = "Estimate",
main = "send_democracy / send_autocracy over the moving window")
axis.Date(1, at = pretty(mid_time), format = "%Y")
abline(h = 0, lty = 2, col = "grey50")
legend("topleft", legend = c("send_democracy", "send_autocracy"),
      col = c("blue", "red"), lty = 1, lwd = 2, cex = 0.8)

```

The support for H2 versus H3 is **not constant**: in periods where the `send_democracy` estimate dips clearly below zero, H2 (democratic restraint *and* autocratic aggression) gains; when it hovers near zero, H3 (autocratic aggression only) takes over. Aggregating over the whole period hides this structure — which is the point of quantifying evidence in windows.

Exercise: test your own constrained hypotheses

The regime-type effects invite other substantive questions. Try replacing `hyp_gov` with each of the following and re-running Step 4 (and, if you like, the moving window). Predict the outcome first, then check.

1. **Is autocratic aggression *stronger, equal, or lower* than democratic effect?** Test the *relative magnitudes* directly:

```

hyp_mag <- "send_democracy < send_autocracy;
           send_democracy = send_autocracy;
           send_democracy > send_autocracy"

```

Perform the test. Which prior probabilities for the hypotheses did you use?

2. **Different mechanisms.** Can we test the two endogenous effects `inertia` and `reciprocity` directly against each other (in other words are they on the same scale?). Formulate different hypotheses on these effects and test them.
3. **An “approximate-equality” (interval) hypothesis.** Ask whether democracy and autocracy effects are *practically equivalent* rather than exactly equal, using an interval constraint. For instance, we could test that the difference does not exceed 0.2. Although the use of `abs()` is not allowed in the `hypothesis` argument, it is possible to write, say, `abs(send_democracy - send_autocracy) < 0.2`, as two one-sided constraints which both need to be satisfied: `send_democracy - send_autocracy < 0.2 & send_autocracy - send_democracy < 0.2`. Try this. Compare the result with the test of the exact equality of the two effects.

For each, report the Bayes factors *and* the posterior probabilities, and state the prior probabilities you used.

Reporting and interpretation checklist

When you write up a BFpack REM analysis, report:

- the **REM specification** (endogenous statistics, covariates, risk set, time scale) and how any covariates were constructed — in particular that the regime dummies are **time-varying**;
- the exact **hypotheses** in constraint notation, with their substantive meaning;
- the **Bayes factors** between hypotheses and the **posterior probabilities**, stating the prior probabilities used;
- for a moving-window analysis, the **window length, step, and burn-in**, and the time axis used; and
- that BF.default uses an **adjusted fractional Bayes factor with a Gaussian approximation**, so reported values are approximate.

Remember that posterior probabilities live on a **continuous** scale — no accept/reject decision is required — and if you *do* commit to a hypothesis, $1 - P(H \mid \text{data})$ is the conditional probability of being wrong given these events and this set of hypotheses.

References

- Gu, X., Mulder, J., & Hoijsink, H. (2018). Approximated adjusted fractional Bayes factors: A general method for testing informative hypotheses. *British Journal of Mathematical and Statistical Psychology*, 71(2), 229–261.
- Lakdawala, R., Leenders, R., & Mulder, J. (2026). Not all bonds are created equal: Dyadic latent class models for relational event data. *Social Networks*, 87, 135–151.
- Meijerink-Bosman, M., Back, M., Geukes, K., Leenders, R., & Mulder, J. (2023). Discovering trends of social interaction behavior over time: An introduction to relational event modeling. *Behavior Research Methods*, 55, 997–1023.
- Mulder, J., & Leenders, R. T. A. J. (2019). Modeling the evolution of interaction behavior in social networks: A dynamic relational event approach for real-time analysis. *Chaos, Solitons & Fractals*, 119, 73–85.
- Mulder, J., et al. (2021). BFpack: Flexible Bayes factor testing of scientific expectations in R. *Journal of Statistical Software*, 1–63.
- Singer, J. D. (1972). The “Correlates of War” project: Interim report and rationale. *World Politics*, 24(2), 243–270.
-